

Cloud Computing Laboratory Report



Name: GuoLinjia

ID: 202383910005

1 Traffic Shifting Experiment

This experiment demonstrates how to gradually migrate traffic from one version of a microservice to another by using Istio traffic management. In the Bookinfo sample application, the reviews service is routed from reviews:v1 to reviews:v3. The experiment first routes all traffic to reviews:v1, then splits traffic equally between reviews:v1 and reviews:v3, and finally routes all traffic to reviews:v3.

1.1 Experiment Objectives

- . Deploy and access the Bookinfo sample application through the Istio ingress gateway.
- . Define service subsets for the reviews service by using DestinationRule.
- . Use VirtualService to route 100% of reviews traffic to reviews:v1.
- . Use weighted routing to split traffic 50% to reviews:v1 and 50% to reviews:v3.
- . Complete the migration by routing 100% of reviews traffic to reviews:v3.

1.2 Experimental Environment

Item	Configuration
Operating system	Windows 11 with Docker Desktop
Kubernetes platform	Minikube cluster
Service mesh	Istio 1.27.1
Sample application	Istio Bookinfo
Access method	kubectl port-forward to http://127.0.0.1:8080/productpage

1.3 Experimental Principle

Istio separates traffic control from application code. DestinationRule defines service subsets according to labels, such as version v1, v2, and v3. VirtualService defines request routing rules and can distribute traffic among different subsets by assigning weights. In this experiment, reviews:v1 does not call the ratings service, so no star ratings are displayed. reviews:v3 calls the ratings service, so red star ratings are shown. Therefore, the page display can be used to verify whether the traffic is routed to v1 or v3.

1.4 Initial Access to the Bookinfo Application

After Istio and Bookinfo were deployed, the application was accessed through the local forwarded address. The initial page was loaded successfully. At this stage, the reviews section was served by reviews:v1, and no star ratings were shown.

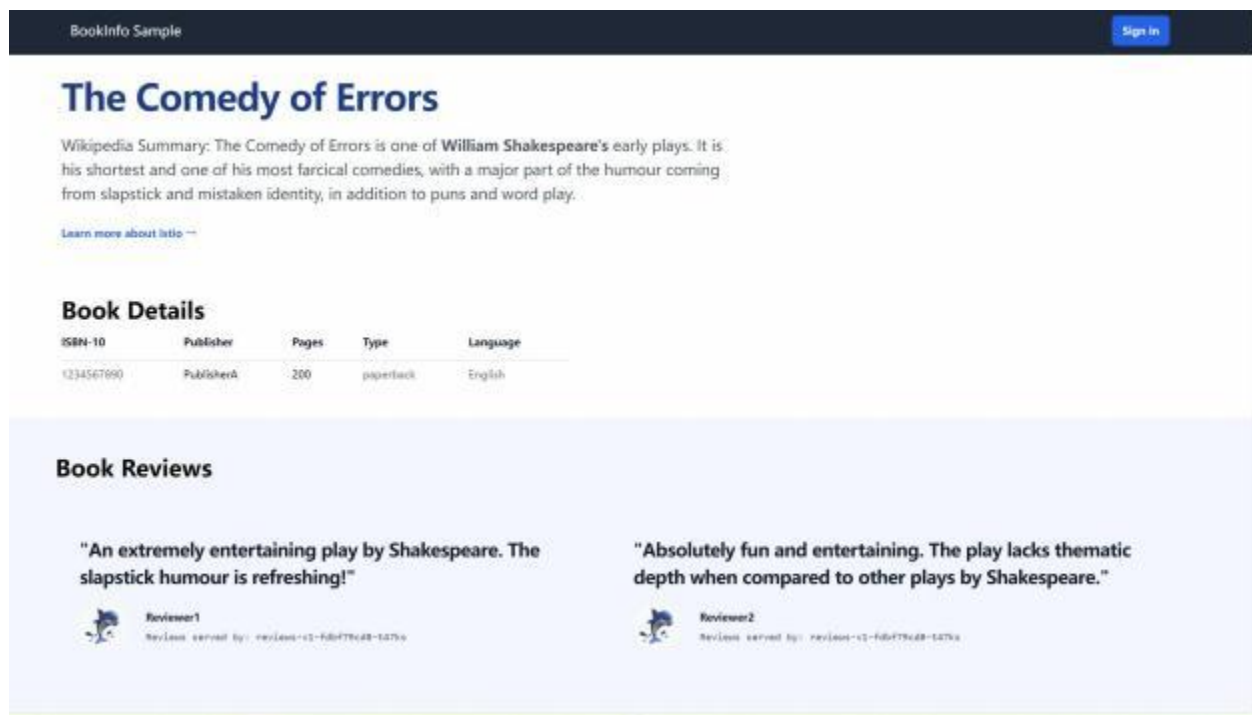


Figure 1. Initial access to the Bookinfo product page served by reviews:v1.

1.5 Defining Service Versions with DestinationRule

The following command was used to create DestinationRule resources for the Bookinfo services. The reviews DestinationRule contains three subsets: v1, v2, and v3. These subsets allow the VirtualService to route requests to specific versions of the reviews service.

```
kubectl apply -f samples/bookinfo/networking/destination-rule-all.yaml
kubectl get destinationrule
kubectl get destinationrule reviews -o yaml
```

```
PS D:\Istio\istio-1.27.1> kubectl apply -f samples/bookinfo/networking/destination-rule-all.yaml
destinationrule.networking.istio.io/productpage created
destinationrule.networking.istio.io/reviews created
destinationrule.networking.istio.io/ratings created
destinationrule.networking.istio.io/details created
PS D:\Istio\istio-1.27.1> kubectl get destinationrule
NAME      HOSTS          AGE
details   details        4s
productpage productpage    4s
ratings   ratings        4s
reviews   reviews       4s
PS D:\Istio\istio-1.27.1> kubectl get destinationrule reviews -o yaml
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  annotations:
    kubernetes.io/last-applied-configuration: |
      [{"apiVersion":"networking.istio.io/v1","kind":"DestinationRule","metadata":{"annotations":{"kubernetes.io/last-applied-configuration":"[{"apiVersion\":\"networking.istio.io/v1\",\"kind\":\"DestinationRule\",\"metadata\":{\"annotations\":{\"kubernetes.io/last-applied-configuration\":\"[...\"}]\"}}]\"}}]}]
  creationTimestamp: "2026-05-23T08:46:24Z"
  generation: 1
  name: reviews
  namespace: default
  resourceVersion: "40820"
  uid: 8999dbb6-41dd-44d7-a854-71b04fe5387d
spec:
  host: reviews
  subsets:
  - labels:
    - version: v1
    name: v1
  - labels:
    - version: v2
    name: v2
  - labels:
    - version: v3
    name: v3
```

Figure 2. DestinationRule resources and reviews service subsets v1, v2, and v3.

1.6 Routing All Traffic to reviews:v1

The first routing rule directed all traffic of the reviews service to subset v1. The command created VirtualService resources for the Bookinfo services. The reviews VirtualService showed that the destination subset was v1.

```
kubectl apply -f samples/bookinfo/networking/virtual-service-all-v1.yaml
kubectl get virtualservice reviews -o yaml
```

```
PS D:\istio\istio-1.27.1> kubectl apply -f samples/bookinfo/networking/virtual-service-all-v1.yaml
virtualservice.networking.istio.io/productpage created
virtualservice.networking.istio.io/reviews created
virtualservice.networking.istio.io/ratings created
virtualservice.networking.istio.io/details created
PS D:\istio\istio-1.27.1> kubectl get virtualservice reviews -o yaml
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"networking.istio.io/v1","kind":"VirtualService","metadata":{"annotations":{},"name":"reviews","namespace":"default"},"spec":{"hosts":["reviews"],"http":[{"route":[{"destination":{"host":"reviews","subset":"v1"}}]}]}}
  creationTimestamp: "2024-05-23T08:47:26Z"
  generation: 1
  name: reviews
  namespace: default
  resourceVersion: "48948"
  uid: 8e2df1d1-af8b-423d-b4fb-3134bd11a3d7
spec:
  hosts:
  - reviews
  http:
  - route:
    - destination:
        host: reviews
        subset: v1
```

Figure 3. VirtualService configuration routing all reviews traffic to subset v1.

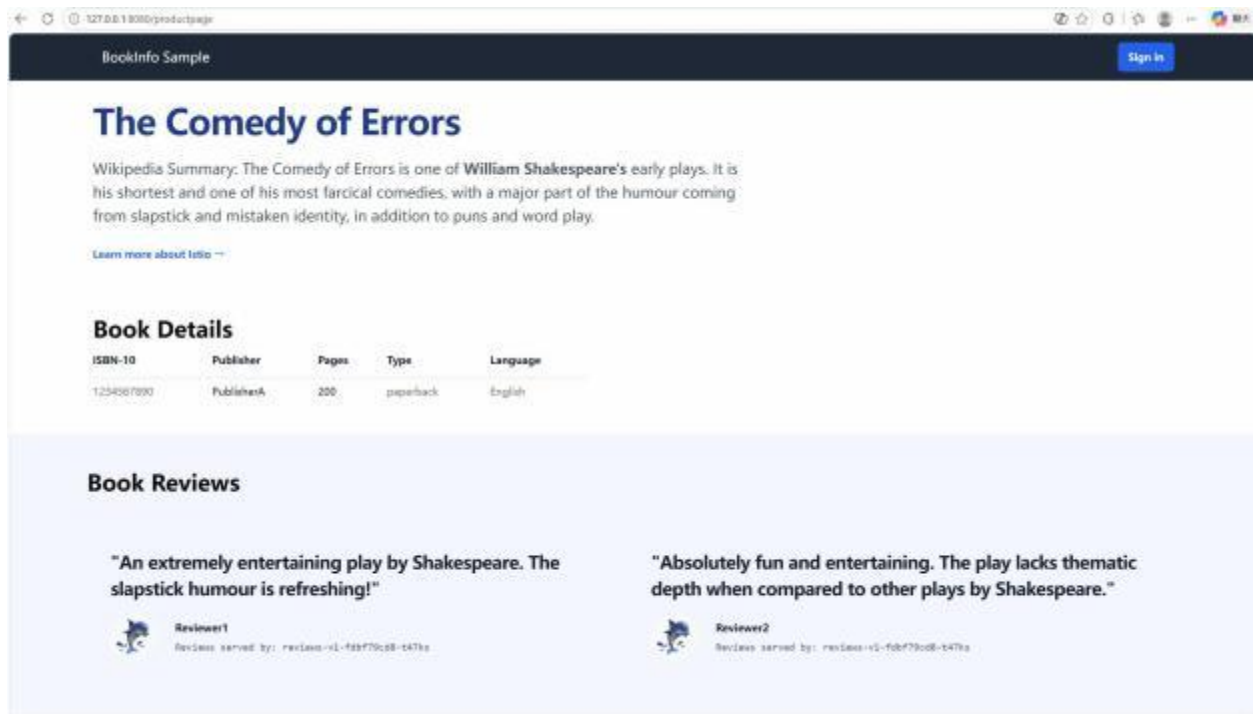


Figure 4. Page result when all traffic is routed to reviews:v1: no star ratings are displayed.

1.7 Applying 50% Traffic Shifting to reviews:v3

Next, weighted routing was applied so that 50% of traffic was sent to reviews:v1 and 50% was sent to reviews:v3. The VirtualService output shows two route destinations with equal weights.

```
kubectl apply -f samples/bookinfo/networking/virtual-service-reviews-50-v3.yaml
kubectl get virtualservice reviews -o yaml
```

```

PS D:\istio\istio-1.27.1> kubectl apply -f samples/bookinfo/networking/virtual-service-reviews-v3.yaml
virtualservice.networking.istio.io/reviews configured
PS D:\istio\istio-1.27.1> kubectl get virtualservice reviews -o yaml
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"networking.istio.io/v1","kind":"VirtualService","metadata":{"annotations":{},"name":"reviews","namespace":"default"},"spec":{"hosts":["reviews"],"http":[{"route":[{"destination":{"host":"reviews","subset":"v1"},"weight":50},{"destination":{"host":"reviews","subset":"v3"},"weight":50}]}]}}
  creationTimestamp: "2026-05-23T08:47:26Z"
  generation: 2
  name: reviews
  namespace: default
  resourceVersion: "41132"
  uid: 8e2df141-af0b-423d-b6fb-3136bd11a3d7
spec:
  hosts:
  - reviews
  http:
  - route:
    - destination:
        host: reviews
        subset: v1
        weight: 50
    - destination:
        host: reviews
        subset: v3
        weight: 50

```

Figure 5. VirtualService configuration with 50% traffic to reviews:v1 and 50% traffic to reviews:v3.

After the rule propagated, refreshing the product page produced two possible results. When the request reached reviews:v1, the reviews section had no star ratings. When the request reached reviews:v3, red star ratings appeared. This confirms that the weighted traffic shifting rule was effective.

BookInfo Sample

Sign In

The Comedy of Errors

Wikipedia Summary: The Comedy of Errors is one of William Shakespeare's early plays. It is his shortest and one of his most farcical comedies, with a major part of the humour coming from slapstick and mistaken identity, in addition to puns and word play.

[Learn more about Istio →](#)

Book Details

ISBN-10	Publisher	Pages	Type	Language
1234567890	PublisherA	200	paperback	English

Book Reviews

"An extremely entertaining play by Shakespeare. The slapstick humour is refreshing!"



Reviewer1

Reviews served by: reviews-v1-f2d779c08-b676a

"Absolutely fun and entertaining. The play lacks thematic depth when compared to other plays by Shakespeare."



Reviewer2

Reviews served by: reviews-v3-f2d779c08-b676a

Figure 6. 50% traffic shifting result: request served by reviews:v1 with no star ratings.

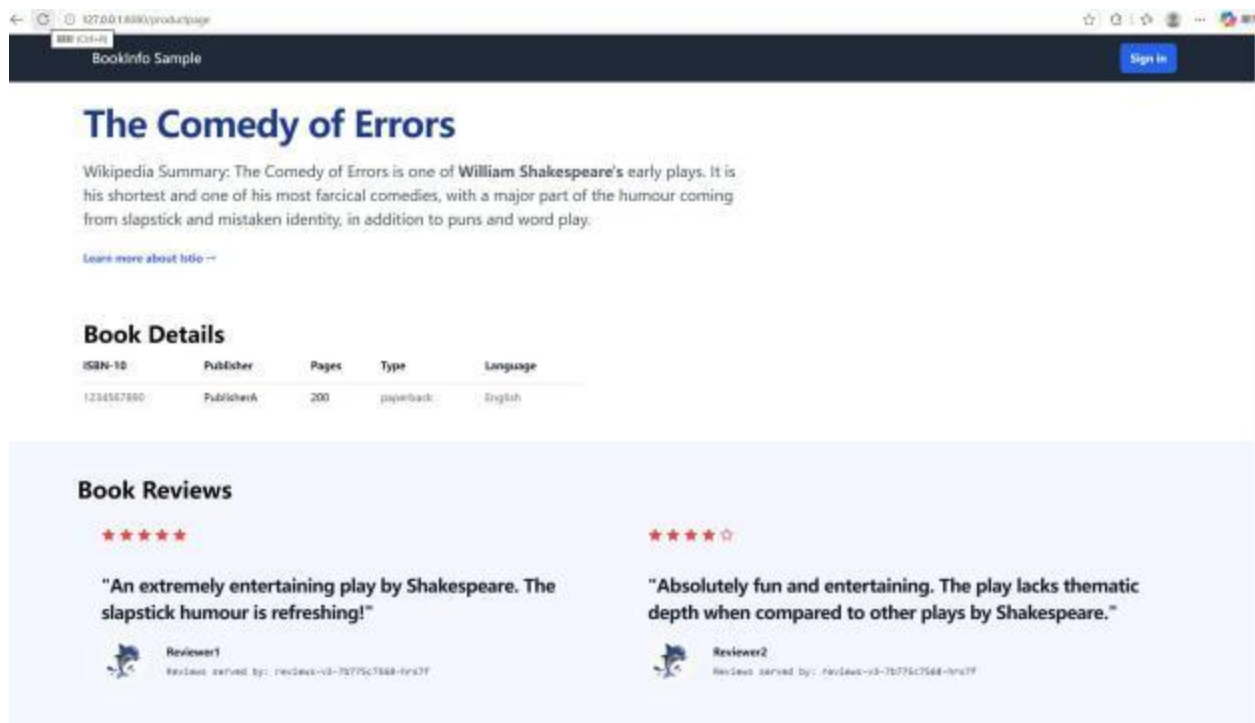


Figure 7. 50% traffic shifting result: request served by reviews:v3 with red star ratings.

1.8 Completing the Migration to reviews:v3

Finally, the routing rule was updated to send all reviews traffic to reviews:v3. The VirtualService output shows only the v3 subset as the destination. After refreshing the page several times, the reviews section consistently displayed red star ratings.

```
kubectl apply -f samples/bookinfo/networking/virtual-service-reviews-v3.yaml
kubectl get virtualservice reviews -o yaml
```

```
PS D:\istio\istio-1.27.1> kubectl apply -f samples/bookinfo/networking/virtual-service-reviews-v3.yaml
virtualservice.networking.istio.io/reviews configured
PS D:\istio\istio-1.27.1> kubectl get virtualservice reviews -o yaml
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      [{"apiVersion": "networking.istio.io/v1", "kind": "VirtualService", "metadata": {"annotations": {}, "name": "reviews", "namespace": "default"}, "spec": {"hosts": ["reviews"], "http": [{"route": [{"destination": {"host": "reviews", "subset": "v3"}}]}}]}]
  creationTimestamp: "2026-05-23T08:47:26Z"
  generation: 1
  name: reviews
  namespace: default
  resourceVersion: "41234"
  uid: 8c2df141-af0b-423d-bd6b-3136bd11a3d7
spec:
  hosts:
  - reviews
  http:
  - route:
    - destination:
        host: reviews
        subset: v3
PS D:\istio\istio-1.27.1>
```

Figure 8. VirtualService configuration routing all reviews traffic to subset v3.

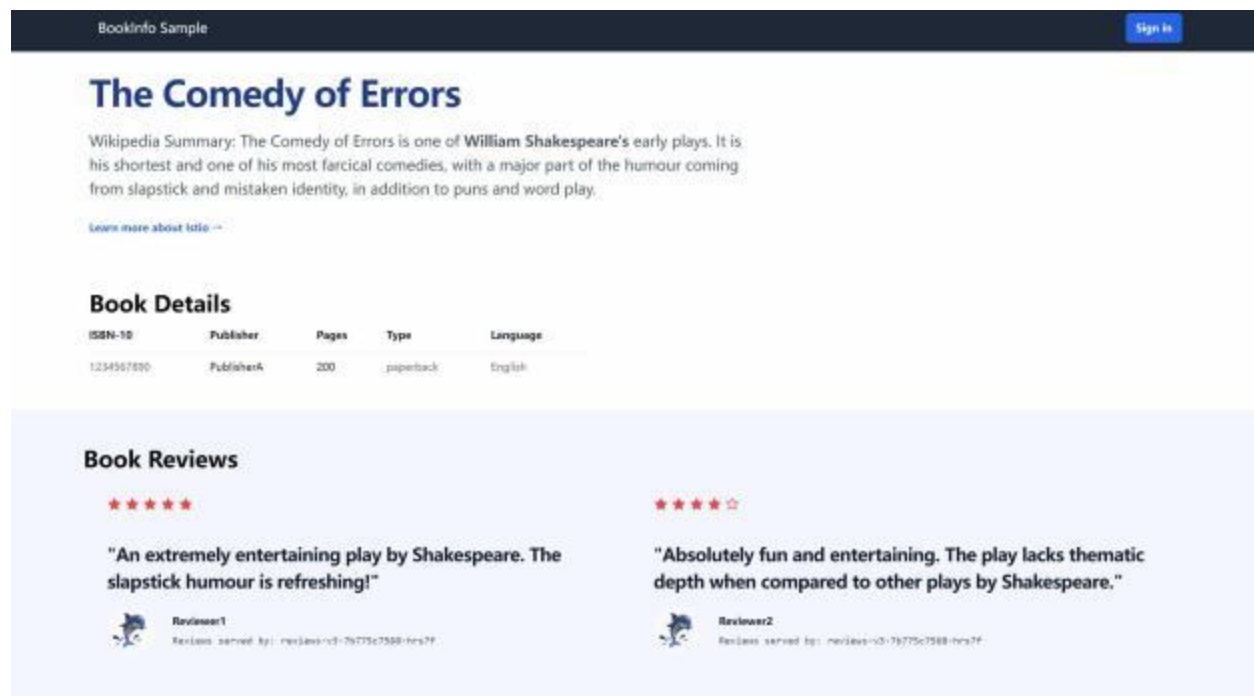


Figure 9. Final result: all reviews traffic is routed to reviews:v3 and red star ratings are always displayed.

1.9 Result Analysis

Stage	Routing rule	Observed page result	Conclusion
Stage 1	100% to reviews:v1	No star ratings	Traffic reached reviews:v1 successfully.
Stage 2	50% to reviews:v1 and 50% to reviews:v3	Both no-star and red-star pages appeared	Weighted traffic shifting worked correctly.
Stage 3	100% to reviews:v3	Red star ratings always appeared	Migration to reviews:v3 was completed.

1.10 Conclusion

This experiment successfully completed traffic shifting in Istio. By configuring DestinationRule and VirtualService resources, traffic was migrated from reviews:v1 to reviews:v3 without changing the application code. The 50% weighted routing stage demonstrated canary-style traffic migration, and the final 100% v3 routing verified that the new version could receive all traffic. Compared with changing the number of service instances, Istio traffic management provides more precise control over request distribution among service versions.

2 Request Routing Experiment

This experiment demonstrates how to dynamically route requests to different versions of a microservice by using Istio traffic management. In the Bookinfo sample application, all services were first routed to version v1. Then, a user-based routing rule was applied so that requests from the user jason were routed to reviews:v2, while requests from other users continued to be routed to reviews:v1.

2.1 Experiment Objectives

- Confirm that the Bookinfo application and Istio components are running correctly.
- Define service subsets for Bookinfo services with DestinationRule.
- Use VirtualService to route all Bookinfo services to version v1.
- Configure user-based routing according to the end-user HTTP header.
- Verify that user jason is routed to reviews:v2 and other users are routed to reviews:v1.

2.2 Experimental Environment and Initial Verification

Before configuring request routing rules, the running status of the Bookinfo application and Istio components was checked. The Bookinfo services were running with sidecar injection enabled, and the Istio control plane and gateways were also running normally.

```
kubectl get pods
kubectl get svc
kubectl get pods -n istio-system
```

```
PS D:\Istio\istio-1.27.1> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
details-v1-6cc9f5cc44-7zgx1        2/2     Running   0           29m
productpage-v1-7f885b46fc-znqcw    2/2     Running   0           29m
ratings-v1-77b8b6df5b-7ndzl       2/2     Running   0           29m
reviews-v1-fdbf79cd8-t47ks         2/2     Running   0           29m
reviews-v2-674c6d8b4-tdtkc        2/2     Running   0           29m
reviews-v3-7b775c7568-hrs7f       2/2     Running   0           29m
PS D:\Istio\istio-1.27.1> kubectl get svc
NAME            TYPE        CLUSTER-IP    EXTERNAL-IP   PORT(S)    AGE
details         ClusterIP   10.98.112.70  <none>        9080/TCP   29m
kubernetes      ClusterIP   10.96.0.1     <none>        443/TCP    39d
productpage     ClusterIP   10.106.255.39 <none>        9080/TCP   29m
ratings         ClusterIP   10.101.26.110 <none>        9080/TCP   29m
reviews         ClusterIP   10.97.86.243  <none>        9080/TCP   29m
PS D:\Istio\istio-1.27.1> kubectl get pods -n istio-system
NAME                                READY   STATUS    RESTARTS   AGE
istio-egressgateway-5fbf54b4bd-94jwg 1/1     Running   0           34m
istio-ingressgateway-77c8b58566-b55rk 1/1     Running   0           34m
istiod-5dc9d7b78-2rq47              1/1     Running   0           34m
PS D:\Istio\istio-1.27.1> |
```

Figure 10. Status of Bookinfo services and Istio components.

The Bookinfo product page was accessed through the forwarded local address. The page loaded successfully, which confirmed that the ingress gateway and Bookinfo gateway were working as expected.

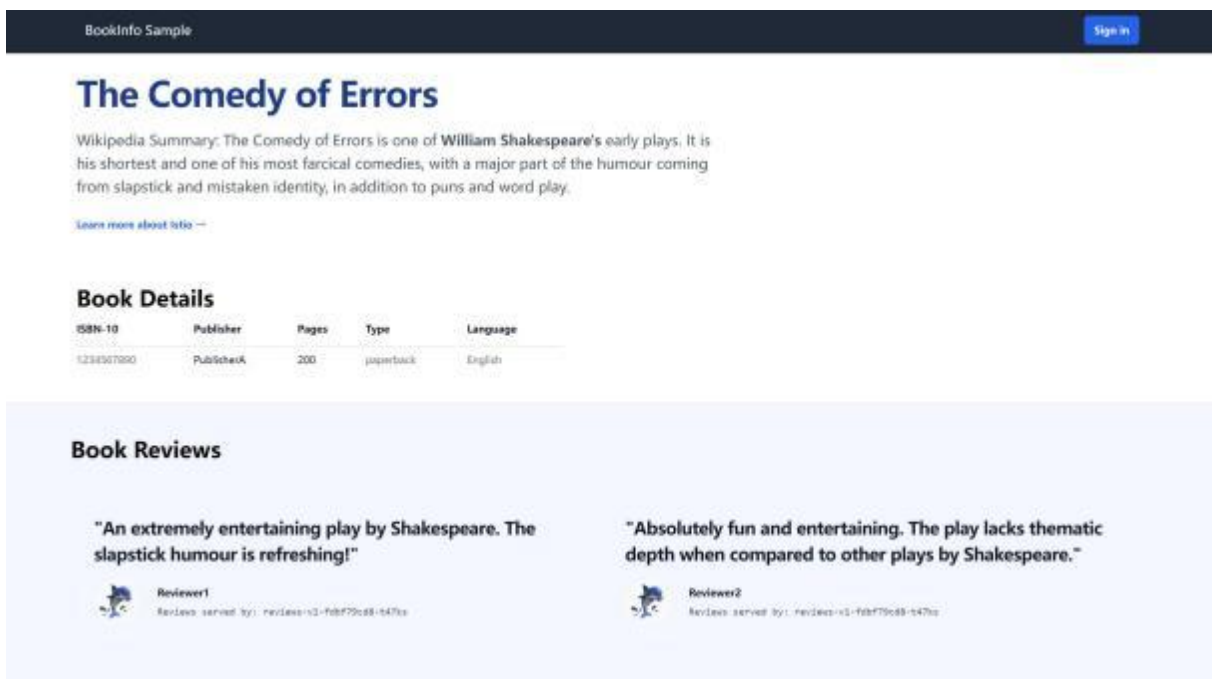


Figure 11. Accessing the Bookinfo product page before request routing verification.

2.3 Defining Service Versions with DestinationRule

The DestinationRule resources were applied to define subsets for the Bookinfo services. For the reviews service, three subsets were available: v1, v2, and v3. These subsets provide the targets used later by VirtualService routing rules.

```
kubectl apply -f samples/bookinfo/networking/destination-rule-all.yaml
kubectl get destinationrule
kubectl get destinationrule reviews -o yaml
```

```
PS D:\istio\istio-1.27.1> kubectl apply -f samples/bookinfo/networking/destination-rule-all.yaml
destinationrule.networking.istio.io/productpage unchanged
destinationrule.networking.istio.io/reviews unchanged
destinationrule.networking.istio.io/ratings unchanged
destinationrule.networking.istio.io/details unchanged
PS D:\istio\istio-1.27.1> kubectl get destinationrules
NAME      HOST      AGE
details    details    28m
productpage productpage 28m
ratings    ratings    28m
reviews    reviews    28m
PS D:\istio\istio-1.27.1> kubectl get destinationrule reviews -o yaml
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      [{"apiVersion": "networking.istio.io/v1", "kind": "DestinationRule", "metadata": {"annotations": {}, "name": "reviews", "namespace": "default"}, "spec": {"host": "reviews", "subsets": [{"labels": {"version": "v1"}, "name": "v1"}, {"labels": {"version": "v2"}, "name": "v2"}, {"labels": {"version": "v3"}, "name": "v3"}]}}]
  creationTimestamp: "2020-09-23T08:46:24Z"
  generation: 1
  name: reviews
  namespace: default
  resourceVersion: "48859"
  uid: 89950b16-01ed-4a17-a854-71b46fa5387d
spec:
  host: reviews
  subsets:
  - labels:
      version: v1
    name: v1
  - labels:
      version: v2
    name: v2
  - labels:
      version: v3
    name: v3
PS D:\istio\istio-1.27.1>
```

Figure 12. DestinationRule definitions for Bookinfo services and reviews subsets.

2.4 Routing All Services to Version v1

The first routing configuration sent traffic to the v1 subset of each Bookinfo microservice. After the VirtualService resources were applied, the route definitions showed that details, productpage, ratings, and reviews all used subset v1 as the destination.

```
kubectl apply -f samples/bookinfo/networking/virtual-service-all-v1.yaml
kubectl get virtualservices -o yaml
```

```
PS D:\istio\istio-1.27.3> kubectl apply -f samples/bookinfo/networking/virtual-service-all-v1.yaml
virtualservice.networking.istio.io/productpage unchanged
virtualservice.networking.istio.io/reviews configured
virtualservice.networking.istio.io/ratings unchanged
virtualservice.networking.istio.io/details unchanged
PS D:\istio\istio-1.27.3> kubectl get virtualservices
NAME          GATEWAYS    HOSTS    AGE
bookinfo     ["bookinfo-gateway"]  ["*"]    39m
details      ["details"]    28m
productpage  ["productpage"]  28m
ratings      ["ratings"]     28m
reviews      ["reviews"]     28m
PS D:\istio\istio-1.27.3> kubectl get virtualservices -o yaml
apiVersion: v1
items:
- apiVersion: networking.istio.io/v1
  kind: VirtualService
  metadata:
    annotations:
      kubectl.kubernetes.io/last-applied-configuration: |
        {"apiVersion":"networking.istio.io/v1","kind":"VirtualService","metadata":{"annotations":{"name":"bookinfo","namespace":"default"},"spec":{"gateways":["bookinfo-gateway"],"hosts":["*"],"http":[{"match":[{"uri":{"exact":"/productpage"}},{uri":{"prefix":"/static"}},{uri":{"exact":"/login"}},{uri":{"exact":"/logout"}},{uri":{"prefix":"/api/v1/products"}]}],route":[{"destination":{"host":"productpage","port":{"number":9080}}}]}}}}
    creationTimestamp: "2026-05-23T08:46:42Z"
    generation: 1
    name: bookinfo
    namespace: default
    resourceVersion: "48702"
    uid: 47b477b8-7bfe-4d91-ad0f-fa5724f94dbb
  spec:
    gateways:
    - bookinfo-gateway
    hosts:
    - "*"
    http:
    - match:
      - uri:
          exact: /productpage
      - uri:
          prefix: /static
      - uri:
          exact: /login
      - uri:
          exact: /logout
    - route:
      - destination:
          host: productpage
          port:
            number: 9080
  apiVersion: networking.istio.io/v1
  kind: VirtualService
  metadata:
    annotations:
      kubectl.kubernetes.io/last-applied-configuration: |
        {"apiVersion":"networking.istio.io/v1","kind":"VirtualService","metadata":{"annotations":{"name":"details","namespace":"default"},"spec":{"hosts":["details"],"http":[{"route":[{"destination":{"host":"details","subset":"v1"}}]}]}}}}
    creationTimestamp: "2026-05-23T08:47:26Z"
    generation: 1
    name: details
    namespace: default
    resourceVersion: "48560"
    uid: 5ca4bdc4-225d-48af-bd45-d36ecf6724d0
  spec:
    hosts:
    - details
    http:
    - route:
      - destination:
          host: details
          subset: v1
  apiVersion: networking.istio.io/v1
  kind: VirtualService
  metadata:
    annotations:
      kubectl.kubernetes.io/last-applied-configuration: |
        {"apiVersion":"networking.istio.io/v1","kind":"VirtualService","metadata":{"annotations":{"name":"productpage","namespace":"default"},"spec":{"hosts":["productpage"],"http":[{"route":[{"destination":{"host":"productpage","subset":"v1"}}]}]}}}}
    creationTimestamp: "2026-05-23T08:47:26Z"
    generation: 1
    name: productpage
    namespace: default
    resourceVersion: "48947"
    uid: 9bc579e9-023e-455a-ba53-f3383d97972e
  spec:
    hosts:
    - productpage
```

Figure 13. Applying VirtualService rules to route Bookinfo services to v1.

```
- uri:
  - exact: /productpage
- uri:
  - prefix: /static
- uri:
  - exact: /login
- uri:
  - exact: /logout
- uri:
  - prefix: /api/v1/products
route:
- destination:
    host: productpage
    port:
      number: 9080
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"networking.istio.io/v1","kind":"VirtualService","metadata":{"annotations":{"name":"details","namespace":"default"},"spec":{"hosts":["details"],"http":[{"route":[{"destination":{"host":"details","subset":"v1"}}]}]}}}}
    creationTimestamp: "2026-05-23T08:47:26Z"
    generation: 1
    name: details
    namespace: default
    resourceVersion: "48560"
    uid: 5ca4bdc4-225d-48af-bd45-d36ecf6724d0
  spec:
    hosts:
    - details
    http:
    - route:
      - destination:
          host: details
          subset: v1
  apiVersion: networking.istio.io/v1
  kind: VirtualService
  metadata:
    annotations:
      kubectl.kubernetes.io/last-applied-configuration: |
        {"apiVersion":"networking.istio.io/v1","kind":"VirtualService","metadata":{"annotations":{"name":"productpage","namespace":"default"},"spec":{"hosts":["productpage"],"http":[{"route":[{"destination":{"host":"productpage","subset":"v1"}}]}]}}}}
    creationTimestamp: "2026-05-23T08:47:26Z"
    generation: 1
    name: productpage
    namespace: default
    resourceVersion: "48947"
    uid: 9bc579e9-023e-455a-ba53-f3383d97972e
  spec:
    hosts:
    - productpage
```

Figure 14. VirtualService output showing the v1 routing configuration.

After refreshing the product page, the reviews section was served by reviews:v1 and no star ratings were displayed. This result was expected because reviews:v1 does not call the ratings service.

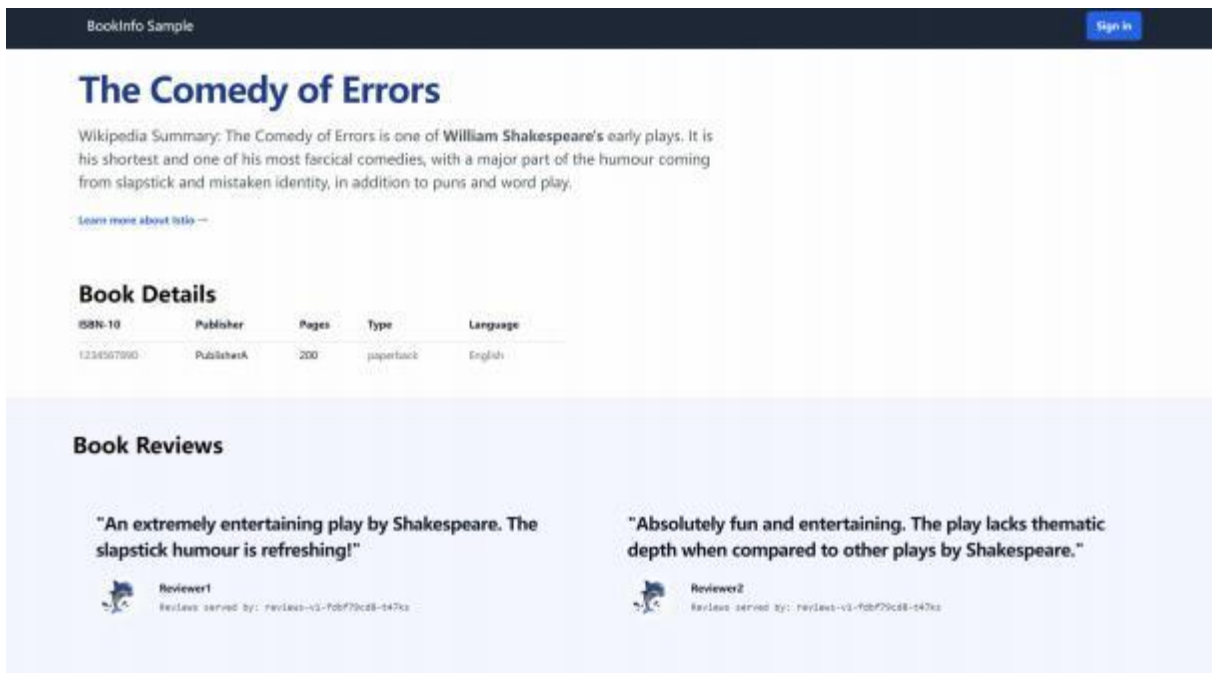


Figure 15. Product page result after routing reviews traffic to v1.

2.5 Routing Based on User Identity

Next, a user-based routing rule was applied. The VirtualService for the reviews service matched the end-user HTTP header. If the header value was jason, the request was routed to reviews:v2. Otherwise, the default route sent the request to reviews:v1.

```
kubectl apply -f samples/bookinfo/networking/virtual-service-reviews-test-v2.yaml
kubectl get virtualservice reviews -o yaml
```

```
PS D:\istio\istio-1.27.1> kubectl apply -f samples/bookinfo/networking/virtual-service-reviews-test-v2.yaml
virtualservice.networking.istio.io/reviews configured
PS D:\istio\istio-1.27.1> kubectl get virtualservice reviews -o yaml
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"networking.istio.io/v1","kind":"VirtualService","metadata":{"annotations":{},"name":"reviews","namespace":"default"},"spec":{"hosts":["reviews"],"http":[{"match":[{"headers":{"end-user":{"exact":"jason"}}}], "route":[{"destination":{"host":"reviews","subset":"v2"}}], [{"route":[{"destination":{"host":"reviews","subset":"v1"}}]}]}]}
  creationTimestamp: "2020-05-23T00:47:20Z"
  generation: 6
  name: reviews
  namespace: default
  resourceVersion: "43794"
  uid: 8e2df1d1-af9b-423d-b6f9-3136bd11a3d7
spec:
  hosts:
  - reviews
  http:
  - match:
    - headers:
        end-user:
          exact: jason
      route:
        destination:
          host: reviews
          subset: v2
    - route:
        destination:
          host: reviews
          subset: v1
PS D:\istio\istio-1.27.1>
```

Figure 16. User-based routing rule that sends user jason to reviews:v2.

2.6 Testing the User-Based Routing Rule

The product page was tested with two different user identities. When the user logged in as jason, the request was routed to reviews:v2 and star ratings were displayed. This verified that the header-based match rule was effective for the specified user.

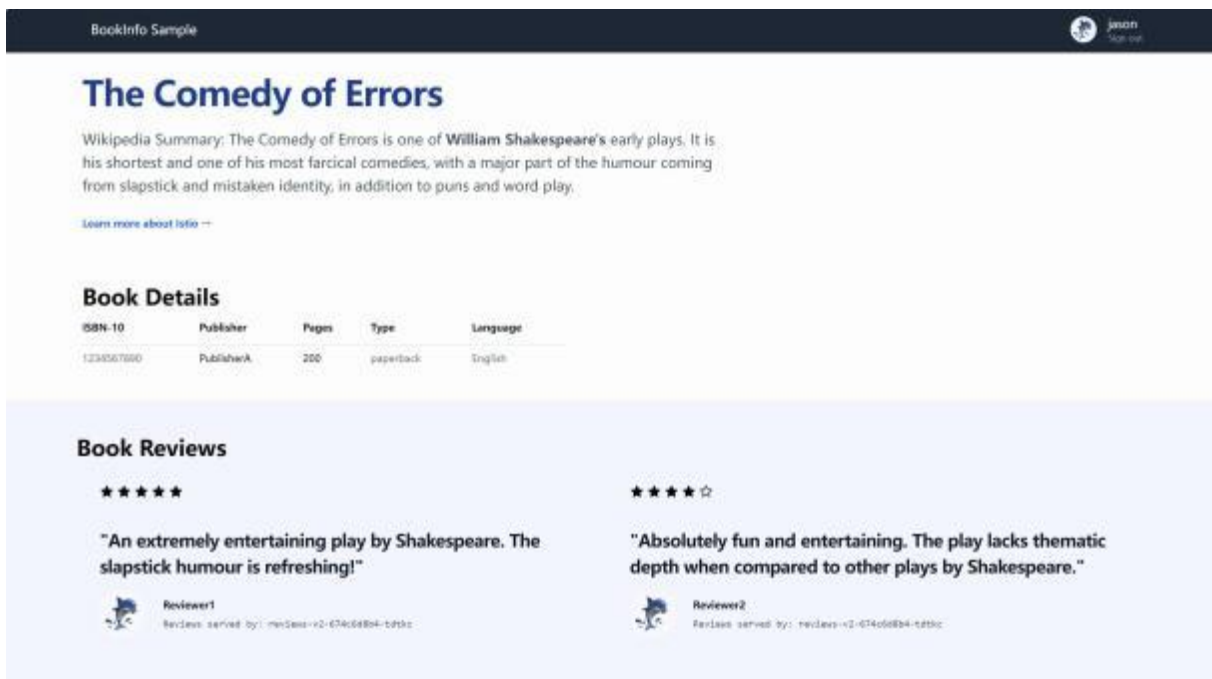


Figure 17. Login as jason: the request is routed to reviews:v2 and star ratings are displayed.

When another user logged in, the request did not match the jason rule. Therefore, the default route was used and traffic was sent to reviews:v1. The page displayed no star ratings, confirming that only user jason was routed to reviews:v2.

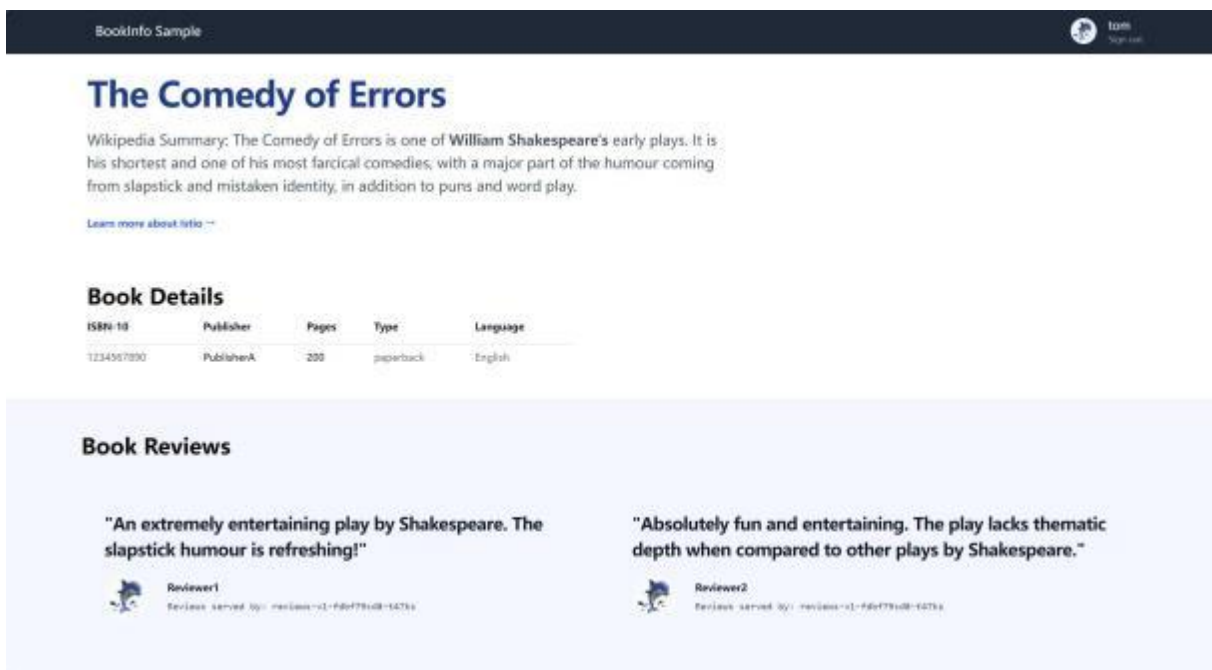


Figure 18. Login as another user: the request is routed to reviews:v1 with no star ratings.

2.7 Result Analysis

The experiment results show that Istio can perform request routing based on both service version and request attributes. The following table summarizes the routing behavior observed in the experiment.

Stage	Routing rule	Observed result	Conclusion
Initial check	Bookinfo and Istio components running	All required pods and services were available	The environment was ready.
Version routing	All services routed to v1	reviews:v1 served the page with no star ratings	Default routing to v1 worked.
User routing	end-user: jason -> reviews:v2	Star ratings appeared after logging in as jason	Header-based routing worked.
Default user route	Other users -> reviews:v1	No star ratings appeared for another user	Only jason was routed to v2.

2.8 Conclusion

This experiment successfully completed request routing in Istio. First, all Bookinfo services were routed to version v1 by applying VirtualService rules. Then, a user-based rule was created for the reviews service. Requests from user jason were routed to reviews:v2, while other users continued to use reviews:v1. The results demonstrate that Istio can manage traffic at the application layer without changing application code, making it suitable for fine-grained routing, testing, and controlled feature exposure.

3 Fault Injection Experiment

This experiment demonstrates how to use Istio fault injection to test the resilience of the Bookinfo application. Based on the previous request routing configuration, requests from user jason are routed to reviews:v2, which calls the ratings service. Two fault injection methods were tested: an HTTP delay fault and an HTTP abort fault. The purpose was to observe how the application behaves when a downstream service becomes slow or returns an error.

3.1 Experiment Objectives

- Restore the request routing configuration in which user jason is routed to reviews:v2 and other users are routed to reviews:v1.
- Inject a 7-second delay fault into the ratings service for user jason.
- Verify that the delay fault causes the product reviews section to become unavailable due to timeout behavior.
- Inject an HTTP 500 abort fault into the ratings service for user jason.
- Verify that the abort fault causes ratings-related error messages while other users remain unaffected.

3.2 Experimental Principle

Istio can inject faults at the service mesh layer without modifying application code. A VirtualService can be configured with a fault field to introduce an artificial delay or abort for matched traffic. In this experiment, the matching condition is the end-user HTTP header with the value jason. Therefore, only the traffic generated by user jason is affected by the fault rule.

In the Bookinfo application, reviews:v2 calls the ratings service. When a delay is injected into ratings, the downstream call becomes slow. Although reviews:v2 has a longer timeout for calling ratings, the productpage service has a shorter timeout when calling reviews. As a result, a 7-second delay causes the product reviews section to fail. In the abort test, ratings directly returns HTTP 500, so the page loads quickly but shows that the ratings service is unavailable.

3.3 Restoring the Request Routing Configuration

Before injecting faults, the request routing rule from the previous experiment was restored. The configuration routes user jason to reviews:v2 and routes all other users to reviews:v1.

```
kubectl apply -f samples/bookinfo/networking/virtual-service-all-v1 .yaml
kubectl apply -f samples/bookinfo/networking/virtual-service-reviews-test-v2.yaml
kubectl get virtualservice reviews -o yaml
```

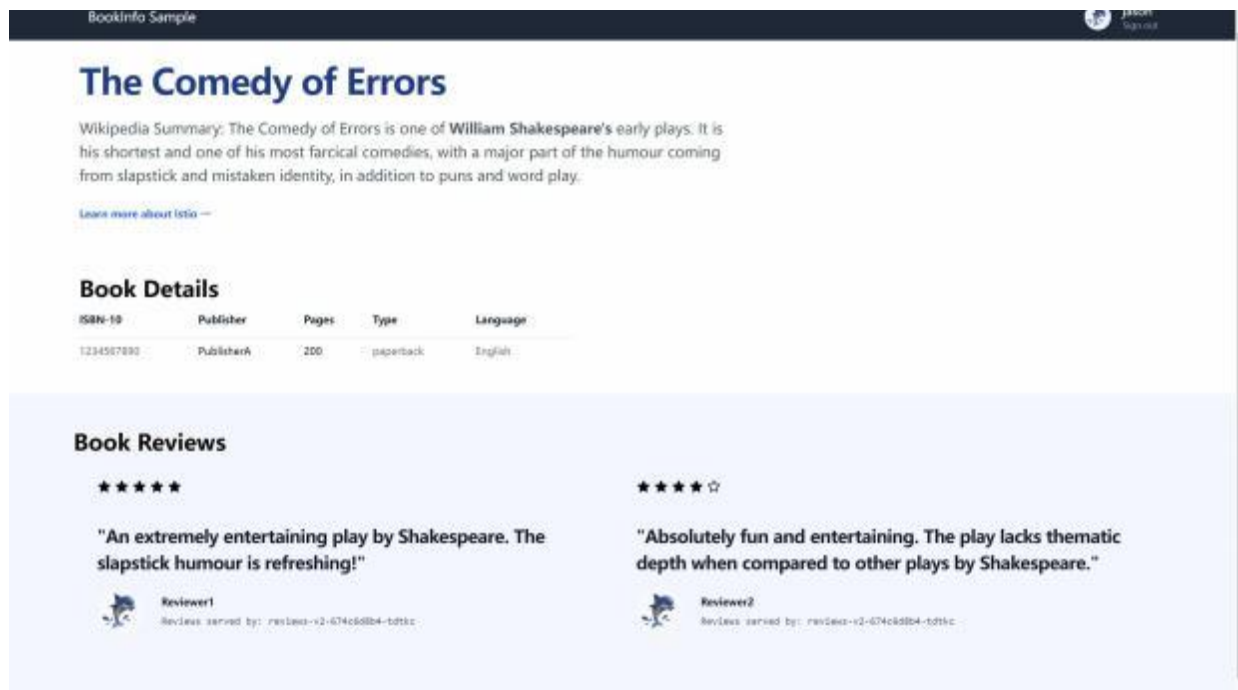


Figure 19. Baseline result before fault injection: user jason is routed to reviews:v2 and star ratings are displayed.

3.4 Injecting an HTTP Delay Fault

A delay fault was injected into the ratings service. The rule applies only when the end-user header is jason. The fixed delay is 7 seconds, and the percentage value is 100, meaning every matched request is delayed.

```
kubectl apply -f samples/bookinfo/networking/virtual-service-ratings-test-delay.yaml
kubectl get virtualservice ratings -o yaml
```

```
PS D:\istio\istio-1.27.1> kubectl apply -f samples/bookinfo/networking/virtual-service-ratings-test-delay.yaml
virtualservice.networking.istio.io/ratings configured
PS D:\istio\istio-1.27.1> kubectl get virtualservice ratings -o yaml
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  annotations:
    kubernetes.io/last-applied-configuration: |
      {"apiVersion":"networking.istio.io/v1","kind":"VirtualService","metadata":{"annotations":{},"name":"ratings","namespace":"default"},"spec":{"hosts":["ratings"],"http":[{"fault":{"delay":{"fixedDelay":"7s","percentage":{"value":100}}},"match":[{"headers":{"end-user":{"exact":"jason"}}},"route":[{"destination":{"host":"ratings","subset":"v1"}}]}]}]}
  creationTimestamp: "2026-09-23T08:49:26Z"
  generation: 2
  name: ratings
  namespace: default
  resourceVersion: "48001"
  uid: 83f462-aa9f-4d6c-a32e-83f144c84eb
spec:
  hosts:
  - ratings
  http:
  - fault:
      delay:
        fixedDelay: 7s
        percentage:
          value: 100
      match:
      - headers:
          end-user:
            exact: jason
      route:
      - destination:
          host: ratings
          subset: v1
    - route:
        destination:
          host: ratings
          subset: v1
PS D:\istio\istio-1.27.1>
```

Figure 20. VirtualService configuration injecting a 7-second delay into the ratings service for user jason.

3.5 Testing the Delay Fault

After the delay rule propagated, the product page was refreshed while logged in as user jason. The reviews section displayed an error message: product reviews were currently unavailable. This indicates that the request path timed out before the delayed ratings response could be completed.

Wikipedia Summary: The Comedy of Errors is one of **William Shakespeare's** early plays. It is his shortest and one of his most farcical comedies, with a major part of the humour coming from slapstick and mistaken identity, in addition to puns and word play.

[Learn more about Istio](#)

ISBN-10	Publisher	Pages	Type	Language
1234567890	PublisherA	200	paperback	English

Sorry, product reviews are currently unavailable for this book.

3.6 Injecting an HTTP Abort Fault

```
kubectl apply -f samples/bookinfo/networking/virtual-service-ratings-test-abort.yaml
kubectl get virtualservice ratings -o yaml
```

```
PS D:\istio\istio-1.27.1> kubectl apply -f samples/bookinfo/networking/virtual-service-ratings-test-abort.yaml
virtualservice.networking.istio.io/ratings configured
PS D:\istio\istio-1.27.1> kubectl get virtualservice ratings -o yaml
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  annotations:
    kubernet.es.io/test-applied-configuration: |
      {"apiVersion": "networking.istio.io/v1", "kind": "VirtualService", "metadata": {"annotations": {}, "name": "ratings", "namespace": "default"}, "spec": {"hosts": ["ratings"], "http": [{"faul
ts": [{"abort": {"httpStatus": 500, "percentage": {"value": 100}}}], "match": [{"headers": {"end-user": {"exact": "jason"}}], "route": [{"destination": {"host": "ratings", "subset": "v1"}}], {"resul
t": [{"destination": {"host": "ratings", "subset": "v1"}}]}]}]
      creationTimestamp: "2024-05-23T08:47:20Z"
      generation: 3
      name: ratings
      namespace: default
      resourceVersion: "48262"
      uid: 83fc4602-aa9f-4c8c-a32e-83f104c99ecb
spec:
  hosts:
  - ratings
  http:
  - fault:
      abort:
        httpStatus: 500
        percentage:
          value: 100
      match:
      - headers:
          end-user:
            exact: jason
      route:
      - destination:
          host: ratings
          subset: v1
  - route:
      - destination:
          host: ratings
          subset: v1
PS D:\istio\istio-1.27.1>
```

3.7 Testing the Abort Fault

GuoLinjia | 202383910005

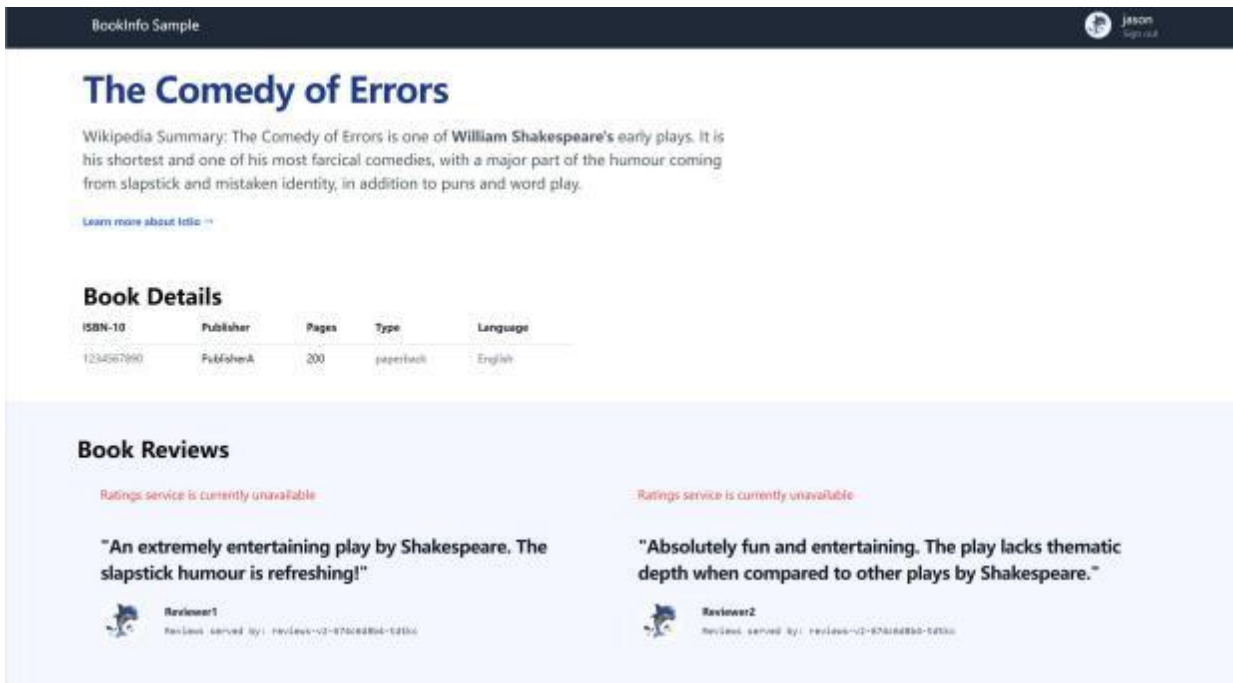


Figure 23. Product page result after injecting the HTTP 500 abort fault.

3.8 Verifying That Other Users Are Not Affected

The fault injection rule only matches traffic with the end-user header value jason. When another user, such as tom, accessed the product page, the request was routed to reviews:v1. Since reviews:v1 does not call the ratings service, the page did not show any ratings-related fault message.

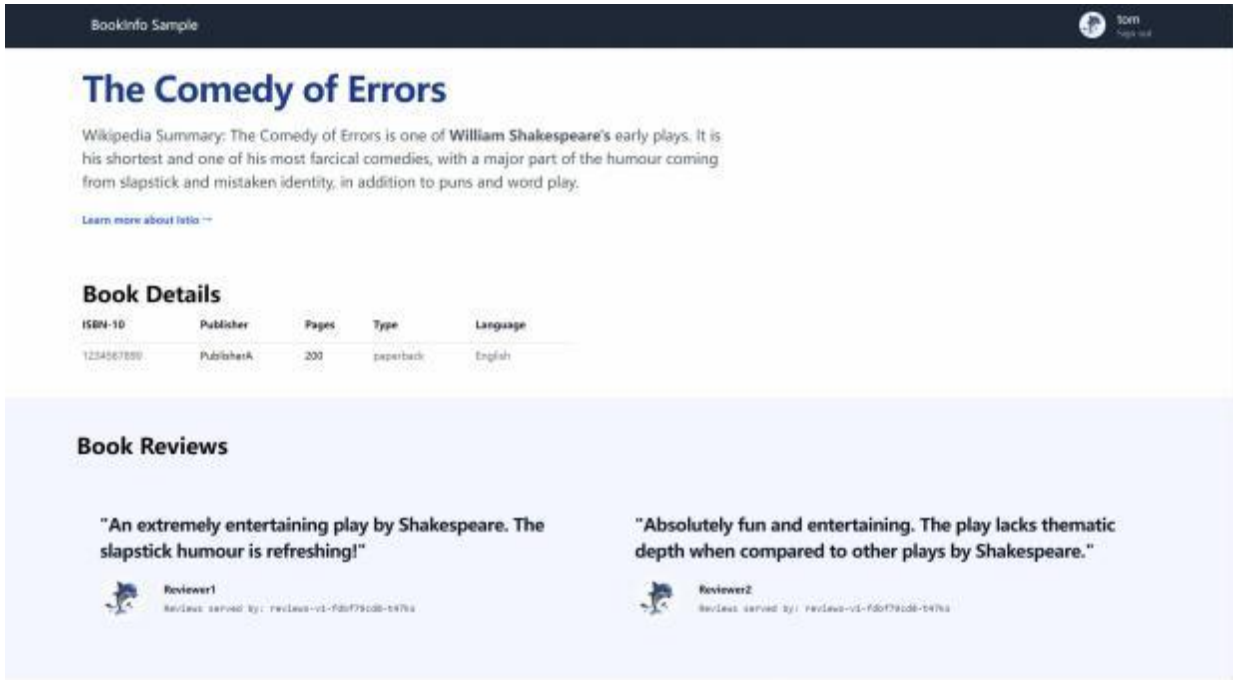


Figure 24. Other users are routed to reviews:v1 and are not affected by the fault injection rule.

3.9 Result Analysis

Stage	Fault rule	Observed result	Conclusion
-------	------------	-----------------	------------

Baseline	jason -> reviews:v2	Star ratings appeared.	Request routing was restored successfully.
Delay fault	7-second delay on ratings for jason	Product reviews became unavailable.	The delay exposed a timeout problem in the request chain.
Abort fault	HTTP 500 abort on ratings for jason	Ratings service unavailable messages appeared.	Istio successfully simulated a downstream service failure.
Other user	No fault match	The page was served by reviews:v1 without error.	The fault injection rule affected only user jason.

3.10 Conclusion

This experiment successfully verified Istio fault injection. By configuring VirtualService rules, a delay fault and an abort fault were injected into the ratings service without changing the Bookinfo application code. The 7-second delay caused the product reviews section to become unavailable, showing that fault injection can expose timeout-related issues between microservices. The HTTP 500 abort fault simulated a direct downstream failure and produced ratings service unavailable messages. Since both fault rules were restricted to user jason, other users were not affected. This demonstrates that Istio can support controlled resilience testing in a microservice system.

4 Circuit Breaking Experiment

This experiment demonstrates how to configure circuit breaking in Istio for service connections, concurrent requests, and outlier detection. The httpbin service was deployed as the backend service, and the Fortio client was used to generate concurrent traffic. By limiting the connection pool and pending requests through a DestinationRule, the experiment intentionally triggered the circuit breaker and observed HTTP 503 responses.

4.1 Experiment Objectives

- Deploy the httpbin backend service with Istio sidecar injection enabled.
- Create a DestinationRule that limits TCP connections and HTTP pending requests for the httpbin service.
- Deploy the Fortio load-testing client and verify normal connectivity to httpbin.
- Increase request concurrency to trigger the circuit breaker.
- Observe HTTP 503 responses and Istio proxy statistics to confirm circuit breaking behavior.
- Clean up the circuit breaking rules and sample services after the test.

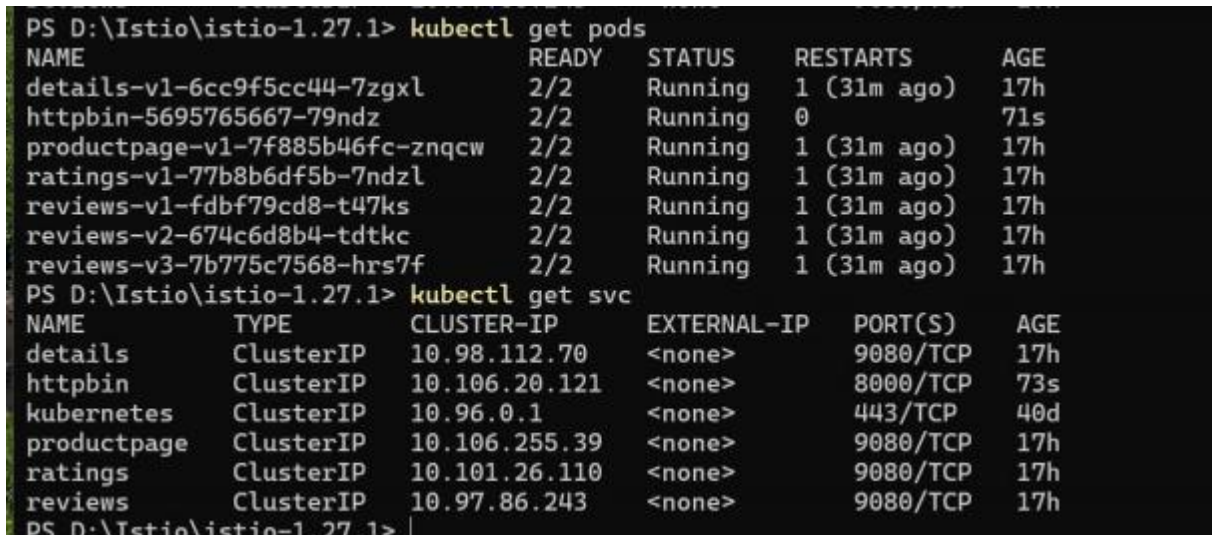
4.2 Experimental Principle

Circuit breaking is a resilience pattern used to prevent a failing or overloaded service from consuming excessive resources. In Istio, circuit breaking can be configured through a DestinationRule. The connectionPool policy limits the number of TCP connections, pending HTTP requests, and requests per connection. The outlierDetection policy can eject unhealthy endpoints after consecutive errors. In this experiment, the httpbin service was configured with maxConnections set to 1, http1MaxPendingRequests set to 1, and maxRequestsPerConnection set to 1. When Fortio generated concurrent traffic beyond these limits, the Istio proxy rejected some requests and returned HTTP 503 responses.

4.3 Deploying the httpbin Backend Service

The httpbin sample service was deployed in the default namespace. Since automatic sidecar injection was enabled, the httpbin pod was injected with an Istio sidecar proxy. The service list shows that httpbin was available on port 8000.

```
kubectl apply -f samples/httpbin/httpbin.yaml
kubectl get pods
kubectl get svc
```



```
PS D:\Istio\istio-1.27.1> kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
details-v1-6cc9f5cc44-7zgxl        2/2     Running   1 (31m ago)  17h
httpbin-5695765667-79ndz           2/2     Running   0           71s
productpage-v1-7f885b46fc-znqcw    2/2     Running   1 (31m ago)  17h
ratings-v1-77b8b6df5b-7ndzl        2/2     Running   1 (31m ago)  17h
reviews-v1-fdbf79cd8-t47ks          2/2     Running   1 (31m ago)  17h
reviews-v2-674c6d8b4-tdtkc          2/2     Running   1 (31m ago)  17h
reviews-v3-7b775c7568-hrs7f        2/2     Running   1 (31m ago)  17h
PS D:\Istio\istio-1.27.1> kubectl get svc
NAME      TYPE        CLUSTER-IP      EXTERNAL-IP  PORT(S)          AGE
details   ClusterIP   10.98.112.70    <none>       9080/TCP         17h
httpbin   ClusterIP   10.106.20.121   <none>       8000/TCP         73s
kubernetes ClusterIP   10.96.0.1       <none>       443/TCP          40d
productpage ClusterIP   10.106.255.39   <none>       9080/TCP         17h
ratings   ClusterIP   10.101.26.110   <none>       9080/TCP         17h
reviews   ClusterIP   10.97.86.243    <none>       9080/TCP         17h
PS D:\Istio\istio-1.27.1>
```

Figure 25. Bookinfo services and the httpbin backend service running in the cluster.

4.4 Configuring the Circuit Breaker DestinationRule

A DestinationRule named httpbin was created to apply circuit breaking settings when clients call the httpbin service. The rule restricts the connection pool and configures outlier detection. The YAML output confirms that the circuit breaker configuration was applied successfully.

```
kubectl apply -f - <<EOF
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  name: httpbin
spec:
  host: httpbin
  trafficPolicy:
    connectionPool:
      tcp:
        maxConnections: 1
      http:
        http1MaxPendingRequests: 1
        maxRequestsPerConnection: 1
    outlierDetection:
      consecutive5xxErrors: 1
      interval: 1s
      baseEjectionTime: 3m
      maxEjectionPercent: 100
EOF
kubectl get destinationrule httpbin -o yaml
```

```
PS D:\istio\istio-1.27.1> #
=> apiVersion: networking.istio.io/v1
=> kind: DestinationRule
=> metadata:
=>   name: httpbin
=> spec:
=>   host: httpbin
=>   trafficPolicy:
=>     connectionPool:
=>       tcp:
=>         maxConnections: 1
=>       http:
=>         http1MaxPendingRequests: 1
=>         maxRequestsPerConnection: 1
=>     outlierDetection:
=>       consecutive5xxErrors: 1
=>       interval: 1s
=>       baseEjectionTime: 3m
=>       maxEjectionPercent: 100
=> # | kubectl apply -f -
destinationrule.networking.istio.io/httpbin created
PS D:\istio\istio-1.27.1> kubectl get destinationrule httpbin -o yaml
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion": "networking.istio.io/v1", "kind": "DestinationRule", "metadata": {"annotations": {}, "name": "httpbin", "namespace": "default"}, "spec": {"host": "httpbin", "trafficPolicy": {"connectionPool": {"http": {"http1MaxPendingRequests": 1, "maxRequestsPerConnection": 1}, "tcp": {"maxConnections": 1}}, "outlierDetection": {"baseEjectionTime": "3m", "consecutive5xxErrors": 1, "interval": "1s", "maxEjectionPercent": 100}}}}
  creationTimestamp: "2026-05-26T02:30:52Z"
  generation: 1
  name: httpbin
  namespace: default
  resourceVersion: "69932"
  uid: 28affb43-d961-48b2-bc42-f6af2b78af43
spec:
  host: httpbin
  trafficPolicy:
    connectionPool:
      http:
        http1MaxPendingRequests: 1
        maxRequestsPerConnection: 1
      tcp:
        maxConnections: 1
    outlierDetection:
      baseEjectionTime: 3m
      consecutive5xxErrors: 1
      interval: 1s
      maxEjectionPercent: 100
```

Figure 26. Creating the DestinationRule for circuit breaking on the httpbin service.

```

metadata:
  annotations:
    kubernetes.io/last-applied-configuration: |
      {"apiVersion": "networking.istio.io/v1", "kind": "DestinationRule", "metadata": {"annotations": {}, "name": "httpbin", "namespace": "default"}, "spec": {"host": "httpbin", "trafficPolicy": {"connectionPool": {"http": {"httpMaxPendingRequests": 1, "maxRequestsPerConnection": 1}, "tcp": {"maxConnections": 1}}, "outlierDetection": {"baseEjectionTime": "3m", "consecutive5xxErrors": 1, "interval": "1s", "maxEjectionPercent": 100}}}}
    creationTimestamp: "2026-05-26T02:30:52Z"
  generation: 1
  name: httpbin
  namespace: default
  resourceVersion: "49933"
  uid: 28a7fb45-d961-4b2-bc42-f6af2b78af45
spec:
  host: httpbin
  trafficPolicy:
    connectionPool:
      http:
        httpMaxPendingRequests: 1
        maxRequestsPerConnection: 1
      tcp:
        maxConnections: 1
    outlierDetection:
      baseEjectionTime: 3m
      consecutive5xxErrors: 1
      interval: 1s
      maxEjectionPercent: 100

```

Figure 27. DestinationRule output showing connection pool and outlier detection settings.

4.5 Deploying the Fortio Load-Testing Client

Fortio was deployed as a client to generate controlled traffic to the httpbin service. The Fortio pod reached the 2/2 Running state, which indicates that both the application container and the Istio sidecar proxy were running.

```

kubectl apply -f samples/httpbin/sample-client/fortio-deploy.yaml
kubectl get pods -l app=fortio

```

```

PS D:\Istio\istio-1.27.1> kubectl apply -f samples/httpbin/sample-client/fortio-deploy.yaml
service/fortio created
deployment.apps/fortio-deploy created
PS D:\Istio\istio-1.27.1> kubectl get pods -l app=fortio
NAME                                READY   STATUS             RESTARTS   AGE
fortio-deploy-78b76b5bdd-j2lfk      1/2     PodInitializing    0           7s
PS D:\Istio\istio-1.27.1> kubectl get pods -l app=fortio
NAME                                READY   STATUS             RESTARTS   AGE
fortio-deploy-78b76b5bdd-j2lfk      1/2     PodInitializing    0           9s
PS D:\Istio\istio-1.27.1> kubectl get pods -l app=fortio
NAME                                READY   STATUS             RESTARTS   AGE
fortio-deploy-78b76b5bdd-j2lfk      1/2     PodInitializing    0          10s
PS D:\Istio\istio-1.27.1> kubectl get pods -l app=fortio
NAME                                READY   STATUS             RESTARTS   AGE
fortio-deploy-78b76b5bdd-j2lfk      1/2     PodInitializing    0          15s
PS D:\Istio\istio-1.27.1> kubectl get pods -l app=fortio
NAME                                READY   STATUS    RESTARTS   AGE
fortio-deploy-78b76b5bdd-j2lfk      2/2     Running   0           63s
PS D:\Istio\istio-1.27.1>

```

Figure 28. Deployment of the Fortio load-testing client with Istio sidecar injection.

The Fortio pod name was then stored in a PowerShell variable so that subsequent commands could execute Fortio inside the client pod.

```

$FORTIO_POD = kubectl get pods -l app=fortio -o
jsonpath="{.items[0].metadata.name}"
echo $FORTIO_POD
PS D:\Istio\istio-1.27.1> $FORTIO_POD = kubectl get pods -l app=fortio -o jsonpath="{.items[0].metadata.name}"
PS D:\Istio\istio-1.27.1> echo $FORTIO_POD
fortio-deploy-78b76b5bdd-j2lfk
PS D:\Istio\istio-1.27.1>

```

Figure 29. Obtaining the Fortio client pod name.

4.6 Verifying Basic Connectivity

Before triggering the circuit breaker, a single request was sent from Fortio to httpbin. The response returned HTTP/1.1 200 OK, confirming that the client could successfully communicate with the backend service under normal conditions.

```

kubectl exec "$FORTIO_POD" -c fortio -- /usr/bin/fortio curl -quiet
http://httpbin:8000/get

```

```

PS D:\istio\istio-1.27.1> kubectl exec "$FORTIO_POD" -c fortio -- /usr/bin/fortio curl -quiet http://httpbin:8000/get
HTTP/1.1 200 OK
Access-Control-Allow-Credentials: true
Access-Control-Allow-Origin: *
Content-Type: application/json; charset=utf-8
Date: Sun, 24 May 2020 02:33:44 GMT
Content-Length: 621
X-Envoy-Upstream-Service-Time: 7
Server: envoy

{
  "args": {},
  "headers": {
    "Host": [
      "httpbin:8000"
    ],
    "User-Agent": [
      "Fortio.org/Fortio-1.69.3"
    ],
    "X-Envoy-Attempt-Count": [
      "1"
    ],
    "X-Forwarded-Client-Cert": {
      "Bynspiffe://cluster.local/ns/default/sa/httpbin:Hash=beee58c80bacf61ef7a4d861321ef4751a4bc192ac3d5118e4458c3c8ea876d;Subject='';URI=spiffe://cluster.local/ns/default/sa/default"
    },
    "X-Forwarded-Proto": [
      "http"
    ],
    "X-Request-Id": [
      "40efc720-f294-95b7-9260-4c16c19171a0"
    ]
  },
  "method": "GET",
  "origin": "127.0.0.1:36441",
  "url": "http://httpbin:8000/get"
}

```

Figure 30. Successful single request from Fortio to the httpbin service.

4.7 Tripping the Circuit Breaker with Concurrent Requests

The circuit breaker was first tested by sending 20 requests with 2 concurrent connections. Because the DestinationRule allowed only one connection and one pending request, some requests were rejected by the proxy. The Fortio output shows both successful 200 responses and failed 503 responses, indicating that the circuit breaker started to take effect.

```

kubectl exec "$FORTIO_POD" -c fortio -- /usr/bin/fortio load -c 2 -qps 0 -n 20 -loglevel Warning http://httpbin:8000/get

```

```

PS D:\istio\istio-1.27.1> kubectl exec "$FORTIO_POD" -c fortio -- /usr/bin/fortio load -c 2 -qps 0 -n 20 -loglevel Warning http://httpbin:8000/get
{"ts":177959083.894083,"level":"info","r":1,"file":"logger.go","line":290,"msg":"Log level is now 3 Warning (max 3 Info)"}
Fortio 1.69.3 running at 0 queries per second, 20-20 procs, for exactly 20 calls (10 per thread + 0)
Starting at max qps with 2 thread(s) (qps=20) for exactly 20 calls (10 per thread + 0)
{"ts":177959083.181088,"level":"warn","r":63,"file":"http_client.go","line":1151,"msg":"Man ok http code", "code":503,"status":"HTTP/1.1 503", "thread":0,"run":0}
{"ts":177959083.115191,"level":"warn","r":63,"file":"http_client.go","line":1151,"msg":"Man ok http code", "code":503,"status":"HTTP/1.1 503", "thread":0,"run":0}
{"ts":177959083.119191,"level":"warn","r":64,"file":"http_client.go","line":1151,"msg":"Man ok http code", "code":503,"status":"HTTP/1.1 503", "thread":1,"run":0}
{"ts":177959083.121569,"level":"warn","r":63,"file":"http_client.go","line":1151,"msg":"Man ok http code", "code":503,"status":"HTTP/1.1 503", "thread":0,"run":0}
{"ts":177959083.123450,"level":"warn","r":64,"file":"http_client.go","line":1151,"msg":"Man ok http code", "code":503,"status":"HTTP/1.1 503", "thread":1,"run":0}
{"ts":177959083.125911,"level":"warn","r":63,"file":"http_client.go","line":1151,"msg":"Man ok http code", "code":503,"status":"HTTP/1.1 503", "thread":0,"run":0}
{"ts":177959083.128299,"level":"warn","r":64,"file":"http_client.go","line":1151,"msg":"Man ok http code", "code":503,"status":"HTTP/1.1 503", "thread":1,"run":0}
{"ts":177959083.130679,"level":"warn","r":63,"file":"http_client.go","line":1151,"msg":"Man ok http code", "code":503,"status":"HTTP/1.1 503", "thread":0,"run":0}
Ended after 33.581742ms : 20 calls, qps=595.56
Aggregated function time : count 20 avg 0.0032448889 +/- 0.002284 min 0.000439853 max 0.0050255 sum 0.064881777
# range, mid point, percentile, count
> 0.000439853 <= 0.001 : 0.000719927 , 30.00, 6
> 0.001 <= 0.003 : 0.0025 , 45.00, 3
> 0.003 <= 0.004 : 0.0035 , 70.00, 5
> 0.004 <= 0.005 : 0.0045 , 85.00, 3
> 0.005 <= 0.006 : 0.0055 , 90.00, 1
> 0.006 <= 0.008 : 0.0075 , 95.00, 1
> 0.008 <= 0.00850256 : 0.00825128 , 100.00, 1
# target 50% 0.0032
# target 75% 0.00433333
# target 90% 0.006
# target 99% 0.0084285
# target 99.9% 0.00949251
Error cases : count 8 avg 0.0012142611 +/- 0.001892 min 0.000439853 max 0.003769949 sum 0.009713689
# range, mid point, percentile, count
> 0.000439853 <= 0.001 : 0.000719927 , 75.00, 6
> 0.001 <= 0.003 : 0.0025 , 87.50, 1
> 0.003 <= 0.00376995 : 0.00338452 , 100.00, 1
# target 50% 0.000775941
# target 75% 0.001
# target 90% 0.0013381
# target 99% 0.00370753
# target 99.9% 0.0037629
# Socket and IP used for each connection:
[0] 5 socket used, resolved to 10.106.28.121:8000, connection timing : count 5 avg 0.0002114388 +/- 0.344e-05 min 0.00010538 max 0.000355145 sum 0.001057194
[1] 4 socket used, resolved to 10.106.28.121:8000, connection timing : count 4 avg 0.00019697825 +/- 6.296e-05 min 0.000124267 max 0.000273396 sum 0.000787913
Connection time (s) : count 9 avg 0.00020501189 +/- 7.637e-05 min 0.00010538 max 0.000355145 sum 0.001845107
Sockets used: 9 (for perfect keepalive, would be 2)
Uniform: false, Jitter: false, Catchup: allowed: true
IP addresses distribution:
10.106.28.121:8000: 9
Code 200 : 12 (60.0 %)
Code 503 : 8 (40.0 %)
Response Header Sizes : count 20 avg 147 +/- 120 min 0 max 245 sum 2940
Response Body/Total Sizes : count 20 avg 616 +/- 306.2 min 241 max 866 sum 12320
All done 20 calls (plus 0 warmup) 3.244 ms avg, 595.6 qps
PS D:\istio\istio-1.27.1>

```

Figure 31. Circuit breaker test result with two concurrent connections.

The concurrency level was then increased to 3 connections and 30 requests. Compared with the previous test, more requests returned HTTP 503. This result shows that a higher concurrency level makes the configured circuit breaker easier to trigger.

```
kubectl exec "$FORTIO_POD" -c fortio -- /usr/bin/fortio load -c 3 -qps 0 -n 30 -loglevel Warning http://httpbin:8000/get
```

```
PS D:\istio\istio-1.27.1> kubectl exec "$FORTIO_POD" -c fortio -- /usr/bin/fortio load -c 3 -qps 0 -n 30 -loglevel Warning http://httpbin:8000/get
[ts:1779598126.984335, "level": "info", "r": 1, "file": "logger.go", "line": 208, "msg": "Log level is now 3 Warning (was 2 Info)"]
Fortio 1.69.5 running at 0 queries per second, 30->10 procs, for 30 calls: http://httpbin:8000/get
Starting at max qps with 3 thread(s) (goxmx 20) for exactly 30 calls (10 per thread + 0)
[ts:1779598126.990651, "level": "warn", "r": 12, "file": "http_client.go", "line": 1151, "msg": "Non ok http code", "code": 503, "status": "HTTP/1.1 503", "thread": 0, "run": 0}
[ts:1779598126.990676, "level": "warn", "r": 13, "file": "http_client.go", "line": 1151, "msg": "Non ok http code", "code": 503, "status": "HTTP/1.1 503", "thread": 1, "run": 0}
[ts:1779598126.991918, "level": "warn", "r": 13, "file": "http_client.go", "line": 1151, "msg": "Non ok http code", "code": 503, "status": "HTTP/1.1 503", "thread": 1, "run": 0}
[ts:1779598126.993176, "level": "warn", "r": 13, "file": "http_client.go", "line": 1151, "msg": "Non ok http code", "code": 503, "status": "HTTP/1.1 503", "thread": 1, "run": 0}
[ts:1779598126.993545, "level": "warn", "r": 14, "file": "http_client.go", "line": 1151, "msg": "Non ok http code", "code": 503, "status": "HTTP/1.1 503", "thread": 2, "run": 0}
[ts:1779598126.994423, "level": "warn", "r": 13, "file": "http_client.go", "line": 1151, "msg": "Non ok http code", "code": 503, "status": "HTTP/1.1 503", "thread": 1, "run": 0}
[ts:1779598126.995745, "level": "warn", "r": 12, "file": "http_client.go", "line": 1151, "msg": "Non ok http code", "code": 503, "status": "HTTP/1.1 503", "thread": 0, "run": 0}
[ts:1779598126.996949, "level": "warn", "r": 13, "file": "http_client.go", "line": 1151, "msg": "Non ok http code", "code": 503, "status": "HTTP/1.1 503", "thread": 1, "run": 0}
[ts:1779598126.997172, "level": "warn", "r": 12, "file": "http_client.go", "line": 1151, "msg": "Non ok http code", "code": 503, "status": "HTTP/1.1 503", "thread": 0, "run": 0}
[ts:1779598126.998184, "level": "warn", "r": 14, "file": "http_client.go", "line": 1151, "msg": "Non ok http code", "code": 503, "status": "HTTP/1.1 503", "thread": 2, "run": 0}
[ts:1779598127.000186, "level": "warn", "r": 14, "file": "http_client.go", "line": 1151, "msg": "Non ok http code", "code": 503, "status": "HTTP/1.1 503", "thread": 2, "run": 0}
[ts:1779598127.001143, "level": "warn", "r": 13, "file": "http_client.go", "line": 1151, "msg": "Non ok http code", "code": 503, "status": "HTTP/1.1 503", "thread": 1, "run": 0}
[ts:1779598127.002670, "level": "warn", "r": 13, "file": "http_client.go", "line": 1151, "msg": "Non ok http code", "code": 503, "status": "HTTP/1.1 503", "thread": 1, "run": 0}
[ts:1779598127.003786, "level": "warn", "r": 12, "file": "http_client.go", "line": 1151, "msg": "Non ok http code", "code": 503, "status": "HTTP/1.1 503", "thread": 0, "run": 0}
[ts:1779598127.004658, "level": "warn", "r": 14, "file": "http_client.go", "line": 1151, "msg": "Non ok http code", "code": 503, "status": "HTTP/1.1 503", "thread": 2, "run": 0}
[ts:1779598127.006947, "level": "warn", "r": 13, "file": "http_client.go", "line": 1151, "msg": "Non ok http code", "code": 503, "status": "HTTP/1.1 503", "thread": 1, "run": 0}
[ts:1779598127.018371, "level": "warn", "r": 14, "file": "http_client.go", "line": 1151, "msg": "Non ok http code", "code": 503, "status": "HTTP/1.1 503", "thread": 2, "run": 0}
[ts:1779598127.012130, "level": "warn", "r": 12, "file": "http_client.go", "line": 1151, "msg": "Non ok http code", "code": 503, "status": "HTTP/1.1 503", "thread": 0, "run": 0}
Ended after 27.026153ms : 30 calls, qps=1078.1
Aggregated Function Time : count 30 avg 0.0023766523 +/- 0.001792 min 0.000379163 max 0.005968074 sum 0.071281568
# range, mid point, percentile, count
> 0.000379163 <= 0.001, 0.000689582, 30.00, 9
> 0.001 <= 0.002, 0.0015, 56.67, 8
> 0.002 <= 0.003, 0.0025, 63.33, 2
> 0.003 <= 0.004, 0.0035, 73.33, 3
> 0.004 <= 0.005, 0.0045, 90.00, 5
> 0.005 <= 0.00596807, 0.00548404, 100.00, 3
# target 50% 0.00175
# target 75% 0.0041
# target 90% 0.005
# target 99% 0.00387127
# target 99.9% 0.00595839
Error cases : count 10 avg 0.001045865 +/- 0.0005628 min 0.000379163 max 0.002101319 sum 0.01082557
```

Figure 32. Fortio load test with three concurrent connections: request logs and latency summary.

```
Ended after 27.026153ms : 30 calls, qps=1078.1
Aggregated Function Time : count 30 avg 0.0023766523 +/- 0.001792 min 0.000379163 max 0.005968074 sum 0.071281568
# range, mid point, percentile, count
> 0.000379163 <= 0.001, 0.000689582, 30.00, 9
> 0.001 <= 0.002, 0.0015, 56.67, 8
> 0.002 <= 0.003, 0.0025, 63.33, 2
> 0.003 <= 0.004, 0.0035, 73.33, 3
> 0.004 <= 0.005, 0.0045, 90.00, 5
> 0.005 <= 0.00596807, 0.00548404, 100.00, 3
# target 50% 0.00175
# target 75% 0.0041
# target 90% 0.005
# target 99% 0.00387127
# target 99.9% 0.00595839
Error cases : count 10 avg 0.001045865 +/- 0.0005628 min 0.000379163 max 0.002101319 sum 0.01082557
# range, mid point, percentile, count
> 0.000379163 <= 0.001, 0.000689582, 30.00, 9
> 0.001 <= 0.002, 0.0015, 56.67, 8
> 0.002 <= 0.003, 0.0025, 63.33, 2
> 0.003 <= 0.004, 0.0035, 73.33, 3
> 0.004 <= 0.005, 0.0045, 90.00, 5
> 0.005 <= 0.00596807, 0.00548404, 100.00, 3
# target 50% 0.00175
# target 75% 0.0041
# target 90% 0.005
# target 99% 0.00387127
# target 99.9% 0.00595839
# Socket and IP used for each connection:
[0] 6 socket used, resolved to 10.106.20.121:8000, connection timing : count 6 avg 0.0002486595 +/- 0.001e-85 min 0.000117874 max 0.000489306 sum 0.001481957
[1] 8 socket used, resolved to 10.106.20.121:8000, connection timing : count 8 avg 0.00016321013 +/- 5.774e-05 min 0.000102382 max 0.000240999 sum 0.001302745
[2] 6 socket used, resolved to 10.106.20.121:8000, connection timing : count 6 avg 0.00015165117 +/- 4.411e-05 min 0.000109648 max 0.000231191 sum 0.000969967
Connection time (s) : count 20 avg 0.00018538045 +/- 7.733e-05 min 0.000102382 max 0.000489306 sum 0.003707609
Sockets used: 20 (for perfect keepalive, would be 3)
Uniform: false, Dither: false, Catchup allowed: true
IP addresses distribution:
10.106.20.121:8000: 28
Code 200 : 12 (40.0 %)
Code 503 : 18 (60.0 %)
Response Header Sizes : count 30 avg 98 +/- 120 min 0 max 245 sum 2940
Response Body/Total Sizes : count 30 avg 491 +/- 306.2 min 241 max 866 sum 14738
All done 30 calls (plus 0 warmup) 2.376 ms avg, 1078.1 qps
PS D:\istio\istio-1.27.1>
```

Figure 33. Fortio load test with three concurrent connections showing HTTP 200 and HTTP 503 results.

4.8 Checking Istio Proxy Circuit Breaker Statistics

The Istio sidecar proxy statistics were queried to further verify the circuit breaking behavior. The output includes `upstream_rq_pending_overflow`, which records requests rejected because the pending request limit was exceeded.

```
kubectl exec "$FORTIO_POD" -c istio-proxy -- pilot-agent request GET stats 2>$null | Select-String "httpbin" | Select-String "pending"
```

```
PS D:\istio\istio-1.27.1> kubectl exec "$FORTIO_POD" -c istio-proxy -- pilot-agent request GET stats 2>$null | Select-String "httpbin" | Select-String "pending"

cluster.outbound[8000][httpbin.default.svc.cluster.local; circuit_breakers.default.remaining_pending: 1
cluster.outbound[8000][httpbin.default.svc.cluster.local; circuit_breakers.default.rq_pending_open: 0
cluster.outbound[8000][httpbin.default.svc.cluster.local; circuit_breakers.high.rq_pending_open: 0
cluster.outbound[8000][httpbin.default.svc.cluster.local; upstream_rq_pending_active: 0
cluster.outbound[8000][httpbin.default.svc.cluster.local; upstream_rq_pending_failure_eject: 0
cluster.outbound[8000][httpbin.default.svc.cluster.local; upstream_rq_pending_overflow: 26
cluster.outbound[8000][httpbin.default.svc.cluster.local; upstream_rq_pending_total: 25
```

Figure 34. Istio proxy statistics showing pending request overflow for httpbin.

4.9 Cleaning Up Circuit Breaking Resources

After the circuit breaking test, the httpbin DestinationRule, the Fortio client, and the httpbin backend service were removed from the cluster.

```
kubectl delete destinationrule httpbin
kubectl delete -f samples/httpbin/sample-client/fortio-deploy.yaml
kubectl delete -f samples/httpbin/httpbin.yaml
```

```
PS D:\Istio\istio-1.27.1> kubectl delete destinationrule httpbin
destinationrule.networking.istio.io "httpbin" deleted from default namespace
PS D:\Istio\istio-1.27.1> kubectl delete -f samples/httpbin/sample-client/fortio-deploy.yaml
service "fortio" deleted from default namespace
deployment.apps "fortio-deploy" deleted from default namespace
PS D:\Istio\istio-1.27.1> kubectl delete -f samples/httpbin/httpbin.yaml
serviceaccount "httpbin" deleted from default namespace
service "httpbin" deleted from default namespace
deployment.apps "httpbin" deleted from default namespace
PS D:\Istio\istio-1.27.1> |
```

Figure 35. Cleanup of the circuit breaking DestinationRule, Fortio client, and httpbin service.

4.10 Result Analysis

The following table summarizes the main observations from the circuit breaking experiment.

Stage	Configuration or action	Observed result	Conclusion
Backend deployment	httpbin service deployed	httpbin pod and service were available.	The backend service was ready for testing.
Circuit breaker rule	maxConnections=1 and http1MaxPendingRequests=1	DestinationRule was applied successfully.	Circuit breaking limits were configured.
Single request	Fortio curl request to httpbin	HTTP/1.1 200 OK was returned.	Normal connectivity was verified.
Concurrent test 1	2 concurrent connections and 20 requests	Both HTTP 200 and HTTP 503 responses appeared.	The circuit breaker started rejecting excessive requests.
Concurrent test 2	3 concurrent connections and 30 requests	More HTTP 503 responses appeared.	Higher concurrency triggered stronger circuit breaking behavior.
Proxy statistics	Query istio-proxy pending metrics	upstream_rq_pending_overflow was shown.	Proxy statistics confirmed pending request overflow.

4.11 Conclusion

This experiment successfully configured and tested circuit breaking in Istio. The httpbin service was protected by a DestinationRule that limited connections and pending requests. A normal single request from Fortio returned HTTP 200, confirming that the service was reachable. When concurrent traffic was increased, some requests returned HTTP 503, showing that the Istio proxy opened the circuit and rejected excessive requests. The proxy statistics further confirmed pending request overflow. This demonstrates how Istio circuit breaking can improve microservice resilience by limiting the impact of overload and preventing excessive requests from overwhelming backend services.